# Timetable Tactics: A Modular SMT Approach to the Exam Timetabling Problem

## 1 Introduction

Effective scheduling of exams is essential in academic institutions, requiring assignments of exams to specific rooms and times while meeting various constraints. This exam timetabling problem is NP-hard and involves satisfying multiple, often conflicting requirements (Müller, 2016). Addressing these challenges, this report presents a solution using Python and the Z3 SMT (Satisfiability Modulo Theories) solver (de Moura and Bjørner, 2008). Additionally, an alternative solution using OR-Tools (Google, 2024) is implemented for comparison, offering a different approach to solving the problem.

The solution leverages Z3's constraint-solving capabilities to handle modular constraints such as room capacities, non-overlapping schedules for students, and invigilator assignments. A graphical user interface (GUI) enables interactive test instance selection, with each instance executed in a subprocess to maintain GUI responsiveness, addressing Z3's limitations with multithreading.

The report further explores the problem formulation, alternative constraint methods, implementation details, and evaluation, alongside potential improvements and broader applications across various scheduling domains.

## 2 Problem Formulation

The exam timetabling problem requires assigning exams to specific rooms and time slots, ensuring that each assignment meets a set of predefined constraints. Let:

- $E = \{e_1, e_2, \ldots, e_n\}$ be the set of exams,
- $R = \{r_1, r_2, \ldots, r_m\}$ be the set of rooms,
- $T = \{t_1, t_2, \ldots, t_k\}$ be the set of ordered time slots,
- $S = \{s_1, s_2, \ldots, s_p\}$ be the set of students,
- $I = \{i_1, i_2, \ldots, i_q\}$ be the set of invigilators,
- $c(r)$ be the capacity of room $r \in R$,
- $s(e) \subseteq S$ represent the set of students assigned to exam $e \in E$.

The following constraints are applied:

1. **Room and Slot Assignment:** Each exam must be assigned to exactly one room and exactly one time slot. Define $\text{Room}(e) \in R$ and $\text{Time}(e) \in T$ as the room and time slot assigned to exam $e$. Then, the assignment constraint can be expressed as:

$$\forall e \in E, \quad \exists r \in R, t \in T : \text{Room}(e) = r \text{ and } \text{Time}(e) = t.$$

2. **Non-Conflicting Room Assignments:** No two exams should be assigned to the same room in the same time slot. For any two exams $e, e' \in E$ (where $e \neq e'$), if $\text{Time}(e) = \text{Time}(e')$, then $\text{Room}(e) \neq \text{Room}(e')$. This constraint is formulated as:

$$\forall e, e' \in E, \quad (\text{Room}(e) = \text{Room}(e') \land \text{Time}(e) = \text{Time}(e')) \Rightarrow e = e'.$$

3. **Room Capacity Constraint:** The number of students assigned to an exam should not exceed the capacity of the room where it is scheduled. For each exam $e$, if $\text{Room}(e) = r$, then $|s(e)| \leq c(r)$. This constraint is expressed as:

$$\forall e \in E, \quad |s(e)| \leq c(\text{Room}(e)).$$

4. **No Consecutive Exams for Students:** Students should not have exams in consecutive time slots. For any two exams $e, e' \in E$ that share at least one student, if $\text{Time}(e) = t$, then $\text{Time}(e') \neq t \pm 1$. This constraint is represented as:

$$\forall e, e' \in E, \quad s(e) \cap s(e') \neq \emptyset \Rightarrow |\text{Time}(e) - \text{Time}(e')| > 1.$$

5. **Invigilator Assignment Constraint:** Each exam must have exactly one invigilator assigned. Define $\text{Invigilator}(e) \in I$ as the invigilator assigned to exam $e$. This requirement is expressed as:

$$\forall e \in E, \quad \exists i \in I : \text{Invigilator}(e) = i.$$

6. **No Consecutive Invigilator Assignments:** To prevent the same invigilator from being assigned to consecutive slots for different exams, for any two exams $e, e' \in E$, if $\text{Invigilator}(e) = \text{Invigilator}(e')$ and $e \neq e'$, then the time slots $t$ and $t'$ assigned to $e$ and $e'$ respectively should not be consecutive. This constraint can be expressed as:

$$\forall e, e' \in E, \quad \text{Invigilator}(e) = \text{Invigilator}(e') \Rightarrow |\text{Time}(e) - \text{Time}(e')| > 1.$$

These constraints are implemented using Z3's SMT functions (Nieuwenhuis et al., 2006), allowing for efficient verification of satisfiability. The modular approach enables constraints to be dynamically applied based on instance attributes.

# 3 Alternative Formulations

While the initial formulations implemented for each constraint ensure the satisfiability of the exam timetabling problem, alternative approaches may simplify or optimise the constraint-solving process. The following alternative formulations address the main constraints while aiming to reduce complexity or enhance readability.

## 3.1 Alternative Formulation for Room and Slot Assignment (Constraint 1 & 2)

The primary formulation uses `ForAll` and `Exists` quantifiers to ensure that each exam is assigned to exactly one room and one slot. In the alternative formulation, we can maintain the same logic but introduce a composite variable $\text{Assignment}(e) = (r, t)$, where $r \in R$ and $t \in T$, to represent both the room and time slot assigned to each exam. The assignment constraint can be expressed as:

$$\forall e \in E, \quad \exists \text{Assignment}(e) \in R \times T : \text{Assignment}(e) = (r, t).$$

This ensures that for each exam $e$, a pair $(r, t)$ is selected from the Cartesian product of the set of rooms $R$ and the set of time slots $T$.

For the non-conflicting room assignments, we ensure that no two exams are assigned to the same room in the same time slot. If $e$ and $e'$ are two distinct exams, and $\text{Assignment}(e) = (r_1, t_1)$ and $\text{Assignment}(e') = (r_2, t_2)$, then if $t_1 = t_2$ (i.e., the exams are assigned to the same time slot), it must hold that $r_1 \neq r_2$ (i.e., they are assigned to different rooms). This constraint is formulated as:

$$\forall e, e' \in E, \quad (\text{Assignment}(e) = (r_1, t_1) \land \text{Assignment}(e') = (r_2, t_2) \land t_1 = t_2) \Rightarrow r_1 \neq r_2.$$

This alternative formulation simplifies the original model by consolidating room and time slot assignments into a single composite variable. However, it may complicate handling more granular constraints on rooms or slots separately, and may also require additional handling for room capacities or other specific room-based conditions (Jovanović and de Moura, 2011).

### 3.2 Alternative Formulation for Room Capacity (Constraint 3)

In the original approach, the room capacity constraint uses implications to check that each exam's student count does not exceed room capacity. An alternative approach could precompute feasible room options based on the student count for each exam. Let:

$$\text{FeasibleRooms}(e) = \{r \in R : |s(e)| \leq c(r)\}.$$

Then, for each exam, the constraint would only assign a room from this precomputed subset:

$$\forall e \in E, \quad \text{Room}(e) \in \text{FeasibleRooms}(e).$$

This reduces the need for runtime checks within the solver, as Z3 only considers feasible room options during its evaluation.

### 3.3 Alternative Formulation for No Consecutive Exams (Constraint 4)

The initial approach prohibits consecutive exams for students by imposing constraints on all exams for each student. As an alternative, we could precompute consecutive exam pairs for each student and apply constraints only to these specific pairs. Let $\text{ConsecutivePairs}(s)$ represent all pairs of exams $(e, e')$ for student $s$ that occur in consecutive time slots. The constraint could then be reformulated as:

$$\forall s \in S, \forall (e, e') \in \text{ConsecutivePairs}(s), \quad |\text{Time}(e) - \text{Time}(e')| > 1.$$

This reduces the number of constraints, focusing only on consecutive slot pairs rather than enforcing this constraint across all possible exams for each student.

### 3.4 Alternative Formulation for Invigilator Assignment (Constraint 5)

The invigilator assignment constraint initially requires each exam to have exactly one invigilator. An alternative approach could involve grouping exams by time slot and assigning invigilators to these groups. Let:

$$\text{AssignedInvigilator}(t) = \{i \in I : i \text{ is assigned to a time slot } t\}.$$

Then for each exam $e$ occurring in time slot $t$, we enforce that:

$$\text{Invigilator}(e) \in \text{AssignedInvigilator}(t).$$

This allows invigilators to be assigned based on time slots rather than on a per-exam basis, reducing the number of constraints and potentially improving performance.

### 3.5 Alternative Formulation for No Consecutive Invigilator Assignments (Constraint 6)

The original constraint prevents invigilators from being assigned to consecutive exams by checking time slot adjacency. An alternative approach could use a sequence-based constraint to track invigilator assignments over consecutive slots directly. Define a variable $\text{NextSlot}(i, t)$ that maps an invigilator $i$ to their next assigned slot following $t$. Then, for each invigilator $i$ assigned to exams $e$ and $e'$ in consecutive time slots $t$ and $t + 1$, we enforce:

$$\text{NextSlot}(i, t) \neq t + 1.$$

This formulation tracks invigilator assignments across time slots in sequence, potentially simplifying adjacency constraints by ensuring that consecutive time slots are never assigned to the same invigilator without needing to check all pairs.

These alternative formulations aim to provide different perspectives on implementing each constraint, potentially improving efficiency or simplifying the solution by reducing the computational load required by Z3.

# 4 Implementation

The implementation of the exam timetabling solution uses Python and the Z3 solver for constraint satisfaction, with a graphical user interface (GUI) enabling users to select problem instances and view the results. A subprocess approach is employed to avoid threading issues with Z3 in multi-threaded GUI environments, allowing multiple instances to be solved concurrently without impacting the GUI's responsiveness. Additionally, the solver finds multiple solutions by iteratively excluding each solution from future checks, allowing for alternative results where possible.

## 4.1 Setup and Execution

The steps to set up and run the solver are as follows:

1. **Create a Virtual Environment:** First, create a virtual environment in Python to ensure dependencies are isolated and compatible. This can be done with:

   ```
   python -m venv env
   ```

2. **Install Required Dependencies:** Install all required packages using the provided `requirements.txt` file:

   ```
   pip install -r requirements.txt
   ```

3. **Run the Solver:** To execute the solver, either of the following main files can be run:

   - `main.py`: Runs the solver without a GUI, processing all instances in the specified folder.
   - `gui.py`: Launches a GUI interface where users can select specific test instances, initiate the solver, and view results in a user-friendly format.

## 4.2 Logic of the Solver

The solver's logic is designed to apply constraints to an instance, find solutions, and output the results. Key steps in this process include instance preparation, constraint application, solution generation, and result formatting.

- **Instance Setup and Declarations:** The solver begins by creating an instance of the `Solver` class in Z3, which will manage all constraints and solution checks. Using the attributes of the instance, the `Declarations` class generates the necessary variables (e.g., exams, rooms, time slots) and functions for constraints. This structure enables constraints to be dynamically applied based on instance attributes.
- **Applying Constraints:** The `ConstraintManager` class manages and applies constraints to the solver. Each constraint is defined in a separate class (e.g., `ExamAssignmentConstraint` or `RoomCapacityConstraint`), with an `add_to_solver` method that adds its specific logic to the solver. This modular design facilitates easy modification and extension of constraints as needed.
- **Constraint Setup and Application:** An array of constraints, such as `RangeConstraint`, `StudentAssignmentsConstraint`, and `ExamAssignmentConstraint`, is set up and managed by `ConstraintManager`, which applies each constraint to the solver in turn. This process ensures that all required constraints are enforced before solution generation.

## 4.3 Detailed Solver Logic

The main logic of the solver is encapsulated in the `solve` function, which operates as follows:

1. **Initialise Solver and Declarations:** A new Z3 solver instance is created, and relevant declarations are generated using the `Declarations` class based on the attributes of the provided instance.

2. **Constraint Setup and Application:** An array of constraints, such as `RangeConstraint`, `StudentAssignmentsConstraint`, and `ExamAssignmentConstraint`, is defined and managed by `ConstraintManager`. Each constraint is applied to the solver in turn.

3. **Solution Loop:** A loop is initiated to check for satisfiability:

   - **Model Extraction:** If the instance is satisfiable, the current model is retrieved and evaluated for the first solution. The assignments of exams to rooms and slots are recorded in `result_text` to display which exam is assigned to each room and slot.
   - **If the user requests for Multiple Solutions:**

     **Exclusion of Current Model:** The current model is then excluded by adding a negation of the current assignments to the solver. This process enables finding additional solutions by ensuring the solver does not return the same model in subsequent checks.

     **Solution Count and Timing:** After each solution, `model_count` is incremented, and the time for finding the first solution is recorded. If the solution count reaches 1,000, the loop terminates.

4. **Result Formatting:** If no solutions are found, `unsat` is recorded in `result_text`. The function then appends the total number of solutions found (or `model_count - 1` for alternatives) and the elapsed times for the first and total solutions. The formatted results are returned to the main execution context or displayed in the GUI.

## 4.4 Additional Constraints Configuration

The solver includes flexibility for customising additional constraints, such as the "Invigilator Mapping" and "No Consecutive Invigilator Assignments" constraint, which ensures that invigilators are not scheduled for consecutive time slots across different exams. Users can enable or disable these constraints through the Graphical User Interface (GUI) or the Command-Line Interface (CLI).

Furthermore, the variables associated with variables governing additional constraints, i.e. `number_of_invigilators` can be modified directly in the `instance.py` file. This allows advanced users to fine-tune the behaviour of the solver to match specific requirements or explore alternative configurations. By adjusting these variables, users can influence the scope and enforcement of the additional constraints without requiring changes to the core solver logic.

## 4.5 Concurrency and GUI Integration

Due to Z3's limitations with threading in GUI applications, the solver uses a subprocess for each instance. This setup prevents blocking the GUI, allowing users to continue interacting with the interface while solutions are computed. The `gui.py` script provides an accessible interface, enabling users to:

- Select one or multiple instances for evaluation.
- Start the solver process and view live results in the GUI's output area.
- View results for each solution, including time elapsed for the first solution and total elapsed time.
- Toggle the generation of multiple solutions for a given instance.
- Toggle the application of additional constraints, i.e. the "Invigilator Mapping" and "No Consecutive Invigilator Assignments" constraints.

This architecture provides flexibility and responsiveness in the user interface, making it easier to work with multiple instances simultaneously without GUI lag. The toggle functionalities enhance usability, giving users control over solution exploration and constraint application directly from the interface.

## 4.6 Multiple Solutions Generation

The solver offers flexibility in generating either a single solution or multiple solutions for the same exam timetabling instance. This functionality is accessible through both the command-line interface (CLI) and the graphical user interface (GUI), which allows users to switch between the two modes based on their preferences or requirements.

- **Single Solution Mode:** When the user selects Single Solution mode, the solver finds and returns the first feasible timetable that satisfies all constraints. This mode is ideal for scenarios where a single, valid solution is sufficient, and no further exploration of alternative timetables is required.
- **Multiple Solutions Mode:** In this mode, the solver employs a systematic method to explore alternative timetables by iteratively excluding each previously found solution. For each solution (or model) that meets all constraints, the solver records the room, slot, and day assignments for each exam. An exclusion constraint is then added, ensuring that at least one exam in the subsequent solution differs in either its room or slot assignment. This prevents the solver from generating identical solutions and instead results in unique, alternative timetables.

## 4.7 Alternate Solver using OR-Tools

As an alternative to the Z3-based implementation, an exam timetabling solver was developed using **OR-Tools**, a Python library for constraint programming by Google. This alternate solver leverages OR-Tools' **Constraint Programming (CP) model** to satisfy the same constraints as the Z3 implementation. The OR-Tools solver is designed for comparison purposes and introduces a modular approach to constraint satisfaction and solution generation.

### 4.7.1 Key Features of the OR-Tools Solver

- **Compatibility with the Problem Structure:** The OR-Tools solver supports a similar input format to the Z3 solver, ensuring compatibility with existing test instances. It reads instance files and initialises decision variables such as exam-room assignments, exam-time assignments, and optional invigilator assignments.
- **Support for Multiple Solutions:** The solver can find multiple solutions by storing each solution and continuing the search until a specified solution count (e.g. 1000) is reached or the user opts for a single solution. This is controlled by a user-configurable flag.
- **Additional Constraints Integration:** The OR-Tools implementation includes the "Invigilator Mapping" and "No Consecutive Invigilator Assignments" constraints, ensuring that invigilators have a valid mapping to exams and are not scheduled for back-to-back time slots for different exams. This feature is optional and can be selected by the user during execution.
- **Scalability and Performance Monitoring:** The solver is capable of handling large instances, providing detailed performance metrics such as the time to find the first solution, total time elapsed, and the number of solutions generated should multiple solutions be enabled.

### 4.7.2 Solver Workflow

1. **Instance Reading and Initialisation:** The solver begins by parsing the problem instance file, initialising the number of exams, rooms, time slots, students, and invigilators. Decision variables for exams, rooms, and time slots are defined using OR-Tools' `NewIntVar` function.
2. **Constraint Definition:** Constraints are added to the model incrementally:
    - **Room and Time Slot Assignment:** Ensure each exam is assigned a room and a time slot.
    - **No Overlapping Exams in Rooms:** Ensures that no two exams are scheduled in the same room at the same time.
    - **Room Capacity:** Verifies that the number of students assigned to a room does not exceed its capacity.
    - **No Consecutive Exams for Students:** Prevents students from being assigned to consecutive exams.
    - **Invigilator Constraints (Optional):** Ensures that each exam has one invigilator and that invigilators are not assigned to consecutive time slots.
3. **Solution Search:** The solver uses the `CpSolver` to explore solutions. A custom `SolutionCounter` callback class is implemented to handle solution extraction and iterative exclusion for multiple solutions.

4. **Result Formatting and Output:** For each solution, the solver outputs the assignments of exams to rooms, time slots, and invigilators (if applicable). Performance metrics such as solution count and timing details are also recorded.

#### 4.7.3 Sample Execution

The solver can be executed via a command-line interface (CLI) with the following configurable options:

- **Single vs. Multiple Solutions:** Users can specify whether to find only the first solution or explore all feasible solutions up to a limit.
- **Invigilator Constraints:** Users can choose to include or exclude invigilator-related constraints.

The OR-Tools solver complements the Z3 implementation, providing an alternative approach to constraint satisfaction with distinct performance characteristics and enhanced flexibility.

The intuitive GUI provides users with control over this process, making it easy to choose the desired mode and review the generated solutions effectively. The ability to toggle between these modes enhances the solver's flexibility, catering to a range of user requirements and scenarios.

# 5 Evaluation

The solver was evaluated using a set of 20 test instances, each containing distinct configurations to assess the solver's performance and accuracy across varied scenarios. The results confirmed that the solver could handle all test instances efficiently, generating correct solutions within acceptable time limits.

## 5.1 Performance and Solution Time

For each test instance, the solution time was tracked and displayed in milliseconds, providing insights into the solver's efficiency. The solver reports the following metrics:

- **First Solution Time:** The time taken to find the first solution for each instance is measured and displayed in milliseconds, allowing a direct view of the solver's initial response time.
- **Total Solution Time:** This metric captures the total time taken to explore all solutions for a given instance. It is especially useful for instances with multiple possible solutions, as it indicates the cumulative time required to identify every possible outcome that satisfies the constraints.
- **Alternative Solutions:** The solver's design allows it to identify alternative solutions by iteratively excluding each current model from further checks. For each instance, the number of alternative solutions is displayed, with an additional indicator if the number of alternatives exceeds a threshold (such as 1,000 solutions).

## 5.2 Correctness and Completeness

Both the primary solver and the alternate solver were tested on the 20 instances with the four mandatory constraints: Room and Slot Assignment, Non-Conflicting Room Assignments, Room Capacity, and No Consecutive Exams for Students. Across all instances, both solvers produced correct results that adhered to each constraint, validating the core functionality of the solution.

Additional testing included the optional invigilator assignment constraints, which were successfully enforced by both solvers on the same instances. This demonstrates their adaptability in handling both mandatory and optional constraints.

The evaluation confirms that both solvers are capable of producing correct and complete solutions that satisfy the requirements of the exam timetabling problem.

## 5.3 GUI Execution and Subprocess Overheads

The solver can also be run through the implemented GUI, providing an accessible way for users to select instances, initiate the solver, and view results in real time. However, testing showed a slight increase in solution time when the solver is run within the GUI. This increase is due to the use of subprocesses to manage each instance independently, ensuring the GUI remains responsive while solving.

Each instance runs as a separate subprocess to prevent threading issues with Z3, which can arise in GUI environments. This subprocess approach, while effective in maintaining the GUI's responsiveness, introduces minor additional overhead due to the need to initialise each subprocess and manage inter-process communication.

## 5.4 Comparison of OR-Tools and Z3 Results

The performance of the solver implementations using OR-Tools and Z3 was evaluated across the same set of test instances under varying configurations. The results are summarised in the table below, highlighting the time elapsed and average time per instance for each configuration:

| Configuration | Framework | Total Time Elapsed (ms) | Average Time Per Instance (ms) |
|---|---|---|---|
| No Multiple, No Additional Constraint | OR-Tools | 535.096 | 26.755 |
| | Z3 | 348.526 | 17.426 |
| Multiple Solutions, No Additional Constraint | OR-Tools | 44701.900 | 2235.095 |
| | Z3 | 27772.905 | 1388.645 |
| No Multiple, Additional Constraint | OR-Tools | 537.498 | 26.875 |
| | Z3 | 323.421 | 16.171 |
| Multiple Solutions, Additional Constraint | OR-Tools | 44794.501 | 2239.725 |
| | Z3 | 13815.707 | 690.785 |

Table 1: Comparison of OR-Tools and Z3 Solver Results

**Key Observations:**

- **Single Solution Scenarios:** Z3 consistently outperformed OR-Tools in both configurations (with and without additional constraints) for single solution generation. Z3 demonstrated significantly lower average times, with up to a 40% reduction in time compared to OR-Tools.
- **Multiple Solution Scenarios:** OR-Tools exhibited substantially higher total elapsed times compared to Z3 in configurations requiring multiple solutions. For example, in scenarios with additional constraints, OR-Tools required over three times the average time per instance compared to Z3, indicating inefficiencies in handling complex solution spaces.
- **Impact of Additional Constraints:** Both frameworks successfully enforced additional constraints, such as "No Consecutive Invigilator Assignments." However, Z3 maintained its performance advantage in both single and multiple solution configurations, while OR-Tools showed a noticeable performance degradation when additional constraints were applied.
- **Efficiency and Scalability:** Z3 demonstrated superior scalability across all configurations, with lower total times and average times per instance. OR-Tools, in contrast, showed poor scalability, especially in configurations with multiple solutions, where the elapsed time increased disproportionately compared to Z3.
- **Algorithmic Efficiency:** The results suggest that Z3's underlying solving algorithms are more efficient in managing constraints and exploring the solution space. OR-Tools struggled particularly in larger solution spaces, where its performance lagged significantly.
- **Use Case Suitability:** Z3's faster computation times and robust handling of additional constraints make it the preferred choice for applications requiring high efficiency and accuracy. OR-Tools' higher computation times and reduced efficiency limit its practicality for scenarios demanding quick and scalable solutions.
- **Reliability in Complex Scenarios:** Z3 proved to be more reliable in managing scenarios with additional constraints and multiple solutions, consistently achieving better performance metrics. OR-Tools' performance in these scenarios highlights its limitations and reinforces Z3's dominance in solving complex constraint satisfaction problems.
- **Overall Solver Preference:** Based on the evaluation, Z3 is the recommended solver for most use cases due to its superior performance, reliability, and ability to handle constraints efficiently. OR-Tools may only be considered when its specific features align with niche requirements, though these cases appear limited based on the results.

### 5.5 Summary of Results

Overall, the solver demonstrated strong performance across all test cases, with Z3 consistently outperforming OR-Tools in both single and multiple solution scenarios. The comparative analysis between the two solvers highlights Z3's significant advantage in efficiency and scalability, particularly in handling constraints and exploring solution spaces under varying configurations.

- **Single Solution Scenarios:** Z3 consistently outperformed OR-Tools in scenarios requiring a single solution, both with and without additional constraints. Z3 demonstrated faster average solution times, achieving 17.426 ms without additional constraints and 16.171 ms with additional constraints, compared to OR-Tools' 26.755 ms and 26.875 ms respectively.
- **Multiple Solution Scenarios:** Z3 outperformed OR-Tools in generating multiple solutions, with an average solution time of 1388.645 ms, approximately twice as fast as OR-Tools' 2235.095 ms for scenarios without additional constraints. For scenarios with additional constraints, Z3 was over three times faster, with a solution time of 690.785 ms compared to OR-Tools' 2239.725 ms. This performance gap can be attributed to OR-Tools' heuristic-based enumeration, which generates solutions sequentially without efficiently excluding previously found ones, leading to slower exploration of larger solution spaces compared to Z3's more direct approach.
- **Impact of Additional Constraints:** Both solvers successfully enforced the "Invigilator Mapping" and the "No Consecutive Invigilator Assignments" constraints. However, Z3 maintained its performance advantage, showing minimal impact on average solution time, while OR-Tools demonstrated a significant increase in elapsed time when additional constraints were applied.
- **Efficiency and Scalability:** Z3 demonstrated superior scalability across all configurations, handling both single and multiple solution scenarios with minimal time overhead. In contrast, OR-Tools struggled with scalability, particularly in configurations requiring multiple solutions, where its performance lagged significantly.
- **GUI Overhead:** Testing through the graphical user interface (GUI) introduced minor overhead due to the subprocess-based execution model. This overhead was negligible for Z3 relative to its overall solution times but compounded OR-Tools' inefficiencies in multiple solution scenarios.
- **Reliability in Complex Scenarios:** Z3 proved to be more reliable and efficient in handling scenarios with additional constraints and multiple solutions. Its ability to maintain low solution times under complex configurations reinforces its dominance as the more robust solver.
- **Overall Solver Preference:** Based on the evaluation, Z3 is the recommended solver for most use cases, offering significantly faster performance, better scalability, and reliable handling of additional constraints. OR-Tools' slower performance and lack of scalability make it less suitable for scenarios requiring efficient and quick solutions, though it can still be used in cases where its feature set aligns with niche requirements.

The comprehensive time-tracking features, alongside detailed performance metrics such as total time elapsed and average time per instance, provide valuable insights into solver behaviour across varied configurations. Z3's consistent performance advantage makes it the clear choice for efficiency-critical applications, while OR-Tools demonstrates limited applicability in handling complex, large-scale problems.

## 6 Discussion

This implementation successfully addresses the complex requirements of the exam timetabling problem, ensuring that all constraints are met while providing a versatile solution adaptable to various scenarios. The solver's modular code structure and GUI interface enhance user interaction, making it easy to select and solve multiple problem instances while viewing results in real time. The use of subprocesses for each instance within the GUI environment effectively circumvents multithreading limitations in Z3, although this requires careful handling of inter-process communication to maintain performance. Z3's efficient constraint satisfaction checking allows for exploration of multiple solutions, showcasing the robustness and flexibility of the solver. Additionally, the alternate solver implemented using OR-Tools serves as a supplementary solution, capable of handling the same constraints and providing correct results under similar scenarios.

## 6.1 Application Overview

The exam timetabling solution is designed to meet the needs of academic institutions and examination boards that must assign exams to specific rooms and time slots while satisfying various constraints, such as room capacities and non-overlapping schedules for students and invigilators. The solver is structured to handle different types of constraints dynamically, allowing for both mandatory and optional conditions based on instance attributes. This adaptability makes it suitable for a wide range of applications, including scenarios where additional requirements (such as invigilator scheduling or custom room allocations) are necessary.

The GUI component is especially valuable for users who may not be familiar with coding, as it provides an intuitive interface for selecting instances, initiating the solver, and viewing results. By offering real-time feedback on solution times and alternate models, the GUI contributes significantly to the overall usability of the application. Additionally, the solver's modularity means that new constraints or adjustments to existing ones can be easily implemented, making the solution extensible and future-proof. This capability is critical for institutions that may need to incorporate changing requirements over time, such as adjusting room capacities or introducing new constraints related to student welfare.

The alternate solver, developed using OR-Tools, further extends the solution's capabilities. It supports the same constraints as the primary solver and is capable of generating correct solutions for exam timetabling problems. While the alternate solver offers comparable functionality in terms of constraint handling, its design and performance provide users with an alternative approach for testing and evaluation.

## 6.2 Extraordinary Features

The solver incorporates several extraordinary features that enhance its robustness, flexibility, and user accessibility. First, the code has been fully refactored and modularised, with each constraint encapsulated in a dedicated class. Utility functions handle additional tasks, such as time tracking and input processing, ensuring that the code remains organised, readable, and maintainable. This modularisation allows for straightforward modifications, enabling constraints to be added or adjusted without affecting the overall structure.

A clear separation of concerns has been implemented throughout the code. For instance, the initialisation of instances is solely managed by the `Instance` class, while attribute reading is delegated to a separate utility function. This separation results in a clean, focused design where each component has a specific role, reducing potential dependencies and making it easier to troubleshoot or expand upon.

Another significant feature is the dynamic initialisation of declarations and constraints. The solver checks for the presence of required attributes in each problem instance, and if specific attributes are absent, related constraints are omitted. This design ensures that the solver only processes relevant constraints, enhancing performance and adaptability to different instance types.

The alternate solver implemented with OR-Tools is also an extraordinary feature. It provides a supplementary approach to solving the exam timetabling problem, capable of handling the same constraints and delivering correct results. This alternative solution is valuable for benchmarking or as a fallback option, offering flexibility in how exam timetabling challenges are addressed.

The GUI implementation adds further value by allowing users to solve single or multiple instances with ease. This graphical interface provides a user-friendly experience, making the solver accessible to individuals who may not have programming experience. Users can select instances, initiate the solver, and view the results directly within the GUI, all while preserving the responsiveness of the interface.

To address multithreading limitations with Z3 in GUI environments, the solver employs a subprocess approach for each instance. By running each instance in its own subprocess, the solver ensures that the GUI remains responsive even when multiple instances are being processed concurrently. Although this adds minor overhead, it significantly enhances the usability of the application.

The GUI also includes toggle functionalities that enhance flexibility and customisation:

- **Multiple Solutions Toggle:** Users can toggle the generation of multiple solutions, enabling the solver to iteratively explore and display alternative feasible timetables. This feature provides insights into the flexibility of scheduling options, catering to scenarios where diverse solutions are desired for comparison or optimisation.

- **Extra Constraint Toggle:** An additional toggle allows users to apply the "No Consecutive Invigilator Assignments" constraint. This feature dynamically adds an extra layer of constraint

logic to ensure that invigilators are not scheduled for consecutive time slots, improving the fairness and practicality of the timetables generated.

Additionally, utility functions have been developed to process input files and track time accurately. These functions allow time differences to be measured in various units, such as milliseconds and seconds, enabling detailed performance evaluation across instances. The solver's capacity to find multiple solutions is also noteworthy; by iteratively excluding each solution from further checks, the solver can display all valid alternatives for each instance, providing insight into the flexibility of possible exam schedules.

# 7 Conclusion

## 7.1 Summary of Contributions

The developed exam timetabling solver effectively addresses the complexities of scheduling under various constraints by leveraging Python and the Z3 SMT solver, complemented by an alternate implementation using OR-Tools. With modular constraint handling, the solver is capable of assigning exams to specific rooms and times while meeting essential requirements, such as room capacity limits, non-overlapping schedules for students, and invigilator assignments. This modular design enables the solver to dynamically adjust constraints based on instance attributes, enhancing adaptability for diverse scheduling needs. The inclusion of the alternate OR-Tools solver demonstrates the flexibility of the implementation and provides an additional approach to solving the timetabling problem.

## 7.2 Limitations

While the solver demonstrates strong performance, certain limitations of Z3 and OR-Tools impact their expressiveness and functionality:

- **Lack of "There Exists Exactly One" ($\exists!$) Operator in Z3:** Z3 does not have a native representation for this operator, requiring workaround formulations that can increase the complexity of constraint modelling.
- **Absence of Universal Set Representation in Z3:** Z3 lacks a direct way to define universal sets for variables. For instance, rather than expressing $e \in E$ directly, a workaround is needed, such as representing it with `Exam_Range(e)` to define the bounds of variable $e$.
- **No Default Mechanism for Multiple Solutions in Z3:** Z3 does not inherently support the retrieval of all possible solutions. An exclusion mechanism is used to iteratively find unique solutions, which may impact performance in instances requiring extensive solution exploration.
- **Performance Constraints in OR-Tools:** While OR-Tools produces correct results for all tested instances, it exhibits significantly higher solution times, particularly in scenarios requiring multiple solutions or additional constraints, limiting its practical usability for large or time-sensitive cases.

## 7.3 Future Work

Potential improvements could focus on the following areas:

- **Integration of Optimisation Techniques:** Incorporating hyper-heuristics or advanced optimisation techniques could enhance scalability and efficiency, particularly for large instances with complex constraints.
- **Exploration of Alternative Solvers:** Investigating additional solvers or extensions that natively address the limitations observed in Z3 and OR-Tools could improve performance and applicability in diverse scheduling scenarios.
- **Enhanced GUI Features:** Expanding the graphical user interface to include features such as detailed visualisations of timetables, conflict resolution suggestions, and interactive constraint adjustments could further improve usability.
- **Adaptive Solver Selection:** Developing a framework that dynamically selects the most suitable solver (e.g., Z3 or OR-Tools) based on instance characteristics could optimise performance across varied scenarios.

- **Application in Broader Domains:** Extending the solver for use in other scheduling problems, such as conference scheduling or workforce rostering, could demonstrate its versatility and encourage wider adoption.

### 7.4 Conclusion

Overall, this exam timetabling solver demonstrates the potential of SMT solvers and constraint programming tools in managing complex scheduling tasks. Through modular design and flexible constraint application, the solution provides a robust and adaptable approach to timetabling. The inclusion of an alternate solver using OR-Tools enhances the comprehensiveness of the solution, offering an additional perspective on addressing scheduling challenges. These contributions establish a strong foundation for future enhancements and pave the way for application in other scheduling domains.

## References

de Moura, L. and Bjørner, N. (2008). Z3: An efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer Berlin Heidelberg.

Google (2024). Or-tools. Accessed: 2024-11-29.

Jovanović, D. and de Moura, L. (2011). Solving non-linear arithmetic. In *Proceedings of the 6th International Joint Conference on Automated Reasoning*, pages 339–354.

Müller, T. (2016). Real-life examination timetabling. *Journal of Scheduling*, 19:257–270.

Nieuwenhuis, R., Oliveras, A., and Tinelli, C. (2006). Solving sat and sat modulo theories: From an abstract davis–putnam–logemann–loveland procedure to dpll(t). *Journal of the ACM (JACM)*, 53(6):937–977.